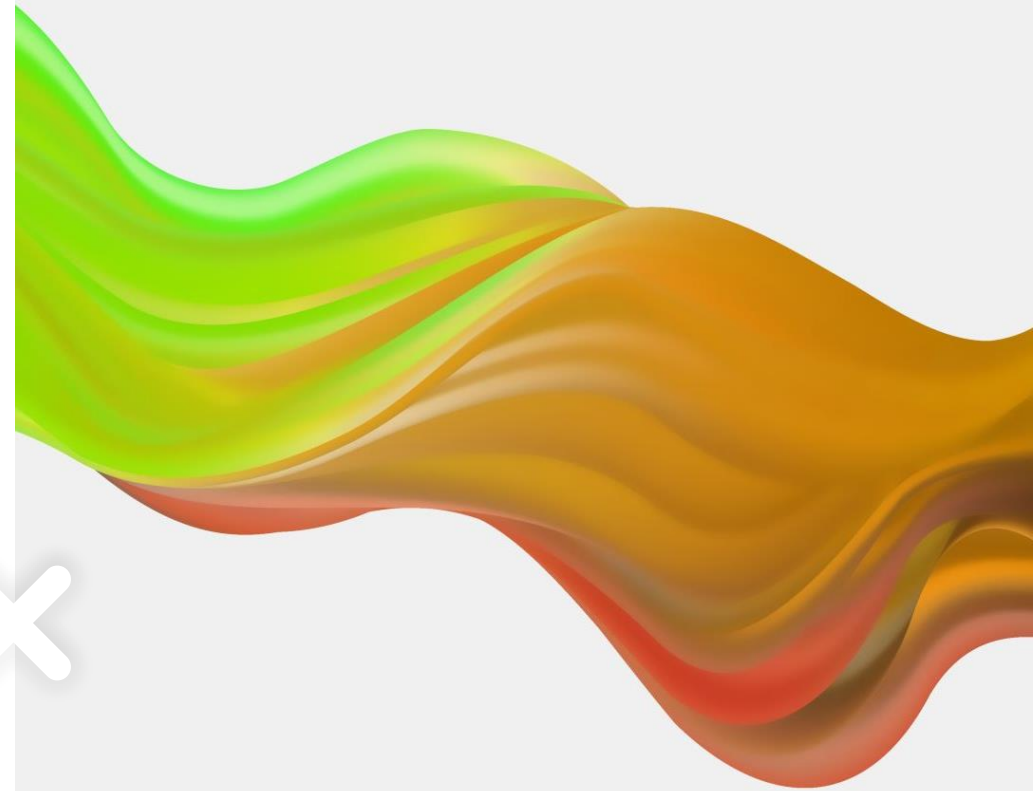




Computer Programming

MENG2020

COPYRIGHT PROTECTED





Introduction



The History of Computing

A close-up, low-angle shot of a hand typing on a laptop keyboard. The lighting is dramatic, with the hand and keys in focus against a dark, blurred background. In the top right corner, there are three white decorative dots of varying sizes and a large white 'X' symbol.

1854

+ **George Boole** creates the algebra that bears his name today (boolean).



AND	T	F
T	T	F
F	F	F

OR	T	F
T	T	T
F	T	F



Computers are electronic systems that can transmit, store and manipulate information (data).

Data can be numeric, character, graphic and sound.

To manipulate data, a computer needs a set of instructions called a program.

Computer Systems



Algorithms

An algorithm is a series of instructions on how to solve the problem. We need algorithms before we can write programs.

Algorithms have the following characteristics:

Precision: must give the correct results.

Uniqueness: same inputs will give the same results.

Finiteness: it must stop eventually.

Input: most algorithms need input.

Output: the answers are the outputs.

Generality: must work with multiple data sets

Introduction - A Look at Digital Computation

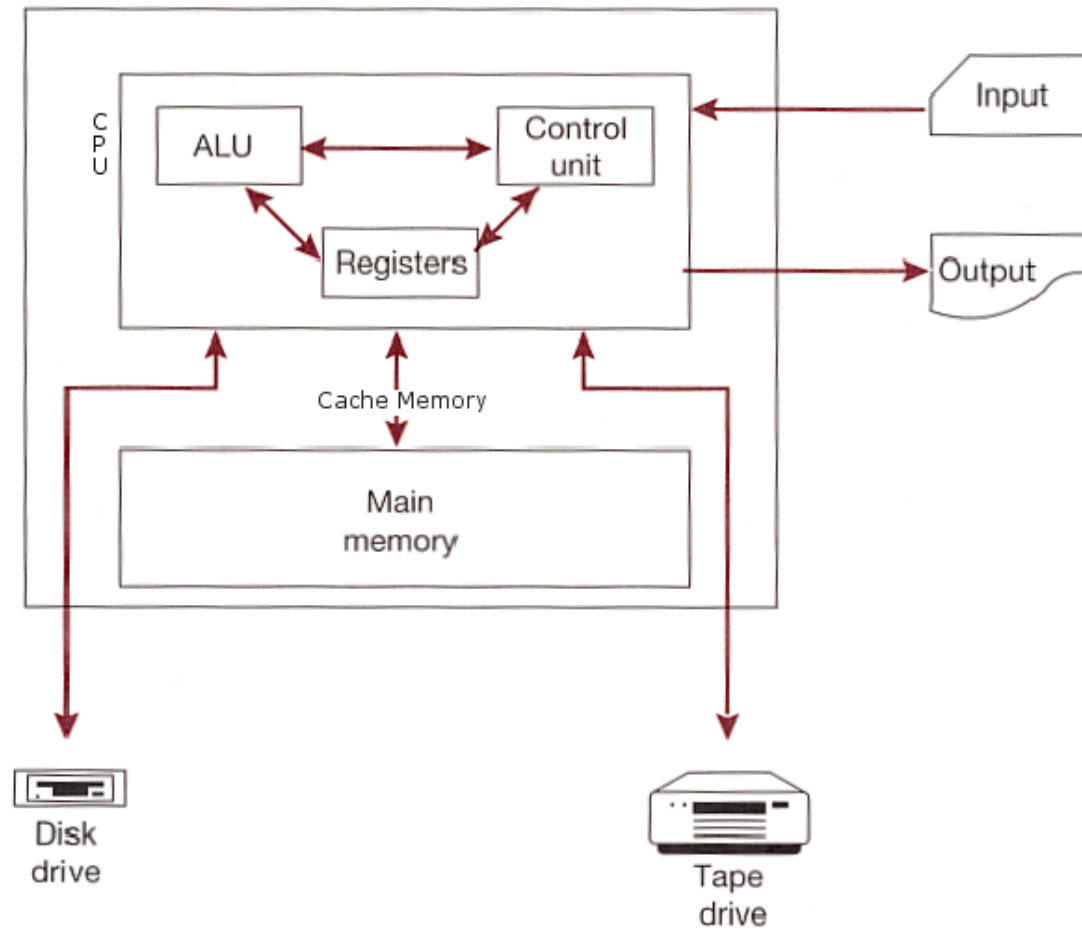
A. Hardware

Physical components that together provide the functionality required by a computing machine

- ❑ CPU (Central Processing Unit)
 - ALU (Arithmetic Logic Unit)
 - Internal Registers
- ❑ Memory
 - RAM (Random Access Memory) aka Read/Write Memory
 - ROM (Read Only Memory)
 - Cache
- ❑ I/O Device
 - Keyboard
 - Mouse
 - Display
 - Printer



Components of a Computer





Storage Types

CPU registers: Only a few cells on CPU.

Read-Only Memory (ROM): Thousands of cells containing pre-set computer instructions.

Main memory (RAM): Billions of cells on circuits separate from CPU. Memory location picked-up at random and access time is the same.

Secondary storage: Hundreds and thousands of billions of cells on disks (magnetic or optic), flash drives or tapes.

Secondary storage is not volatile (data is kept even when power is off).



B. Software

Programming Languages

Procedural

- Pascal
- Fortran
- Basic
- Turing
- C

Object Oriented

- Java
- C++

Applications

- Word Processors
- Spreadsheets
- SAS
- MATLAB

Operating Systems

- Windows
- Unix
- Linux
- MAC-OS



DIGITAL SYSTEMS

Digital systems: Systems that use a binary-like notation,

Number representation can be done in different bases, the most frequently used include:

Base 2 - Binary Number Representation

All numbers are shown using only 2 digits, i.e., 0 and 1

Base 8 - Octal Representation (Oct for short)

All numbers are shown using only 8 digits, i.e., 0, 1, 2, ..., 6, 7

Base 10 - Decimal Representation

All numbers are shown using only 10 digits, i.e., 0, 1, 2, ..., 8, 9

Base 16 - Hexadecimal representation (Hex for short)

All numbers are shown using only 16 digits, i.e., 0, 1, ..., 8, 9, A, B, C, D, E, F



Binary System

To count in binary is similar to the usual count in decimal: You start at 0 and at 9 you run out of digits so you use two instead like 10. In binary we run out of digits at 1 instead of 9. So we count in binary from 0 to 10 decimal with 0, 1, 10, 11, 100, 101, 110, 111, 1000, 1001, and 1010.

Like in decimal, we can pad with zeros to the left without changing the value:
0000 1001 is still a binary equivalent of 9.

Integer numbers shown in different bases

Decimal	Binary	Oct	Hex
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
17	10001	21	11



Internal Representations

Bit: Binary digit (0 or 1).



Byte: A group of 8 bits. One character.



Word: The width of a memory cell.



Nibble: A group of 4 bits. Used to make bit sequences easier to read.



Each byte of main memory has an address.



All numbers and all characters are represented in binary code.

Characters

Characters are expressed using the ASCII code:

'A' = 65 = 01000001

'g' = 103 = 01100111

'\$' = 36 = 00100100

'+' = 43 = 00101011

Note that digits expressed in ASCII are different than integers or doubles.

'9' = 57 = 00111001

See the ASCII code at www.asciitable.com

Programming Languages



Many languages exist to program instructions to the computer.

1st generation: Machine languages (pure binary) Example: *1010111010101010111010101011*

2nd generation: Assembly languages (mnemonic codes)
Example: *MV R1,R3*

3rd generation: High-level languages (C, Fortran, Java)
Example: *y = x * x + 2;*

4th generation: Closer to human languages than typical high-level programming languages (MATLAB, SQL)
Example: *find all records where name is "Kim"*



Solving Problems

▢ Here are the steps required to solve a problem with a computer program:

1. Define the problem
2. Analyze the problem.
3. Design a solution.
4. Implement the solution.
5. Test the program.
6. Update and maintain the program.

Why MATLAB for MENG2020?



It is a very popular software and language with students, engineers, and scientists in universities, research institutes and industries all over the world. It is being used by many advanced courses in the science program.



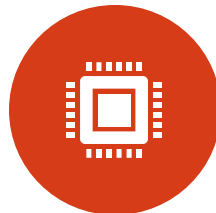
It is powerful and easy to use.



Programs are platform-independent. Programs written on one platform will run on all the other platforms without change (Windows, Unix, Linux, Macintosh).



MATLAB contains an extensive library of predefined functions.



MATLAB has advanced capabilities for plotting and graphics making it a visually attractive environment for new programmers.

MATLAB Advantages



Its basic data element is the matrix (like 2 two-dimensional grid). A simple integer is considered an matrix of one row and one column. Several mathematical operations that work on arrays or matrices are built-in to the Matlab environment. For example, cross-products, dot-products, determinants, inverse matrices.



Vectorized operations. Adding two arrays together needs only one command, instead of a for or while loop.



The graphical output is optimized for interaction. You can plot your data very easily, and then change colors, sizes, scales, etc, by using the graphical interactive tools.



Matlab's functionality can be greatly expanded by the addition of toolboxes. These are sets of specific functions that provided more specialized functionality. Ex: Excel link allows data to be written in a format recognized by Excel, Statistics Toolbox allows more specialized statistical manipulation of data (Anova, Basic Fits, etc)

Matlab Drawbacks

- ▶ It uses a large amount of memory and on slow computers it is very hard to use.
- ▶ It sits “on top” of Windows, getting as much CPU time as Windows allows it to have. This makes real-time applications very complicated.
- ▶ MATLAB is an interpreted language. The main disadvantage of interpreted languages is execution speed.

When a language is compiled, all of the code is analyzed and processed efficiently, before the programmer distributes the application. With an interpreted language, the computer running the program has to analyze and interpret the code before it can be executed (each and every time), resulting in slower processing performance.

- ▶ For applications where performance is critical, a fast language like C is a much better choice.

CPU

CPU has two main parts:

- registers
- ALU

A. Register

Registers are storage units inside the CPU. Registers are used to store the data and the result of operations.

B. ALU

ALU is the unit responsible for 2 types of operations that the CPU performs:

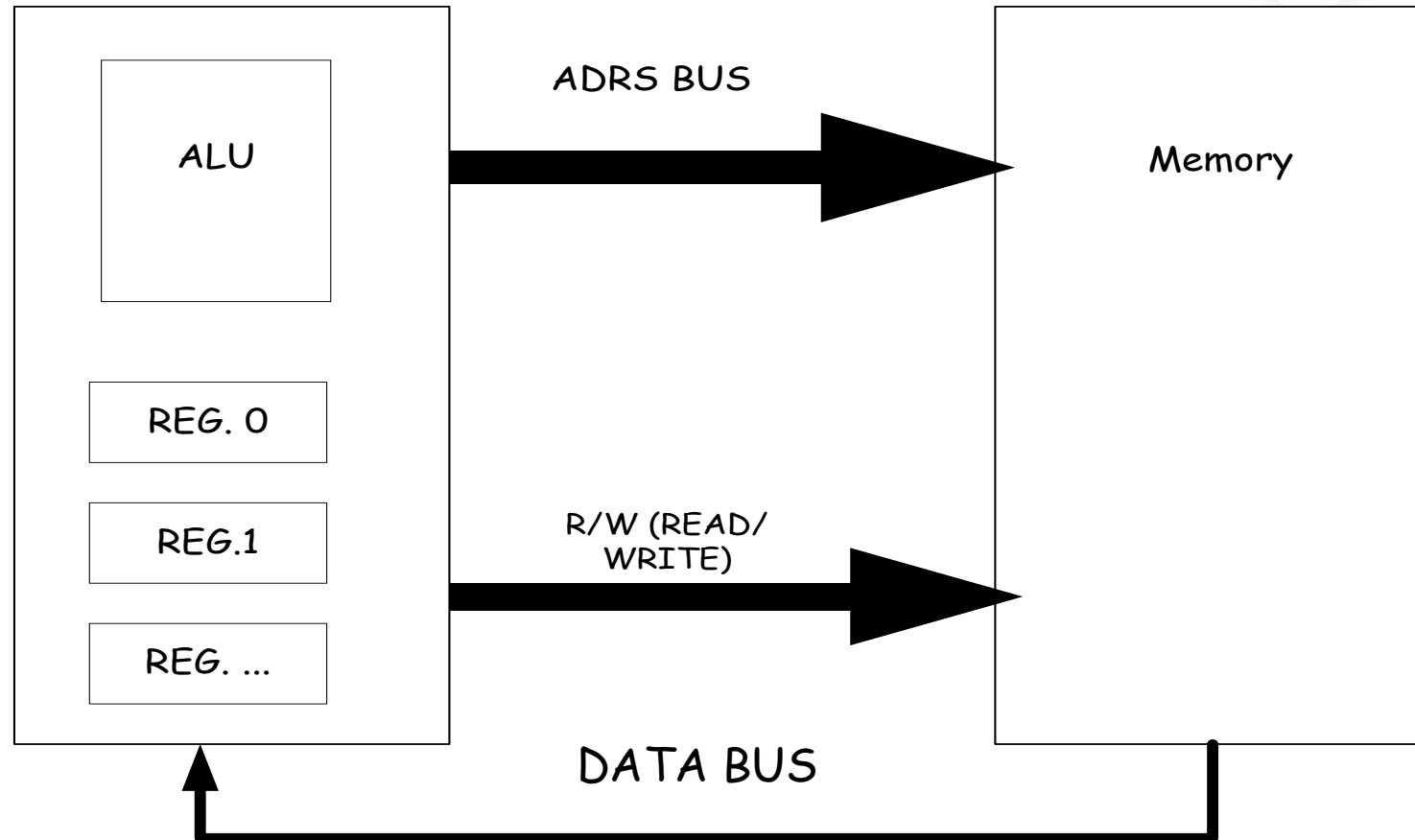
- Arithmetic
- Logical

hence the name ALU (Arithmetic Logical Unit)

Address bus to provide memory address

Data bus to facilitate exchange of data between CPU and memory

Simple interaction between CPU & Memory





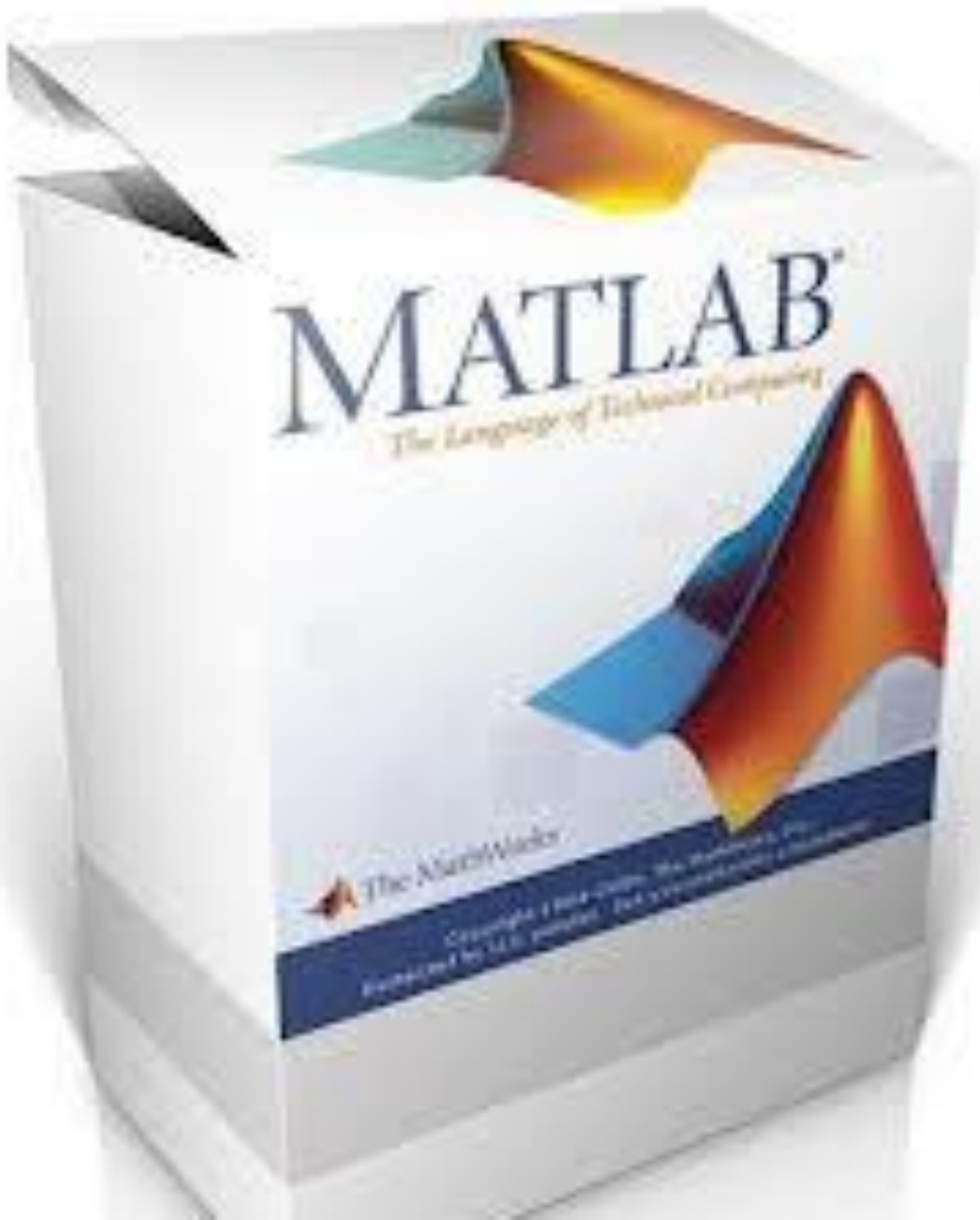
READ:

- CPU puts the address on the address bus
- CPU issues read
- MEMORY puts the data on the data bus
- CPU gets the data from the data bus

READ – WRITE OPERATIONS

WRITE:

- CPU puts the address on the address bus
- CPU puts the data on the data bus
- CPU issues write
- CPU puts the data into the specified memory location

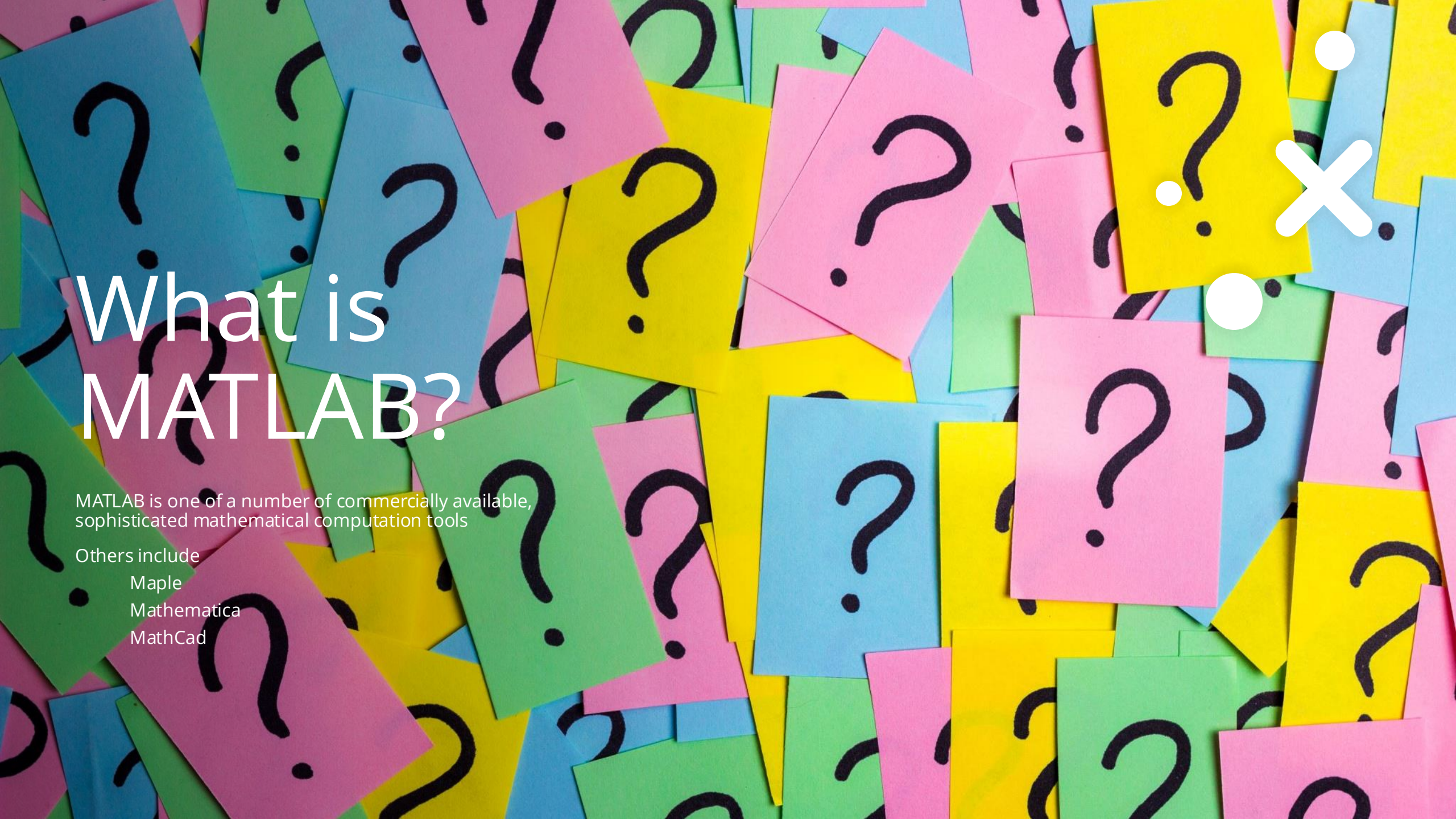


Objectives

Understand what MATLAB is and why it is widely used in engineering and science

Start the MATLAB program and solve simple problems in the command window



The background of the slide is a dense, overlapping collage of colorful sticky notes in shades of blue, green, yellow, and pink. Each sticky note features a large, bold, black question mark. In the upper right corner, there is a white 'X' icon, which is a common symbol for a close or delete button in user interfaces.

What is MATLAB?

MATLAB is one of a number of commercially available, sophisticated mathematical computation tools

Others include

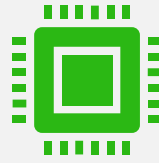
Maple

Mathematica

MathCad



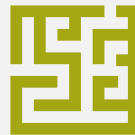
What Is MATLAB?



MATLAB is both a powerful computational environment and a programming language that easily handles matrix and complex arithmetic.



It is a large software package that has many advanced features built in, and it has become a standard tool for many working in science or engineering disciplines.



Among other things, it allows easy plotting in both two and three dimensions.



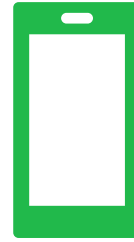
Numerical calculations

- Especially: matrices

Graphics

MATLAB stands for
Matrix Laboratory

MATLAB is
good at:



Easy to use



Built in programming language

Why MATLAB



MATLAB was originally written
in Fortran, then later rewritten
in C



Release Numbers

MATLAB is updated regularly



The Mathworks packages their software in groups, called releases, New releases are issued twice a year in the spring and in the fall

Release 2025 a
includes

MATLAB

A number of
specialized
“toolboxes”

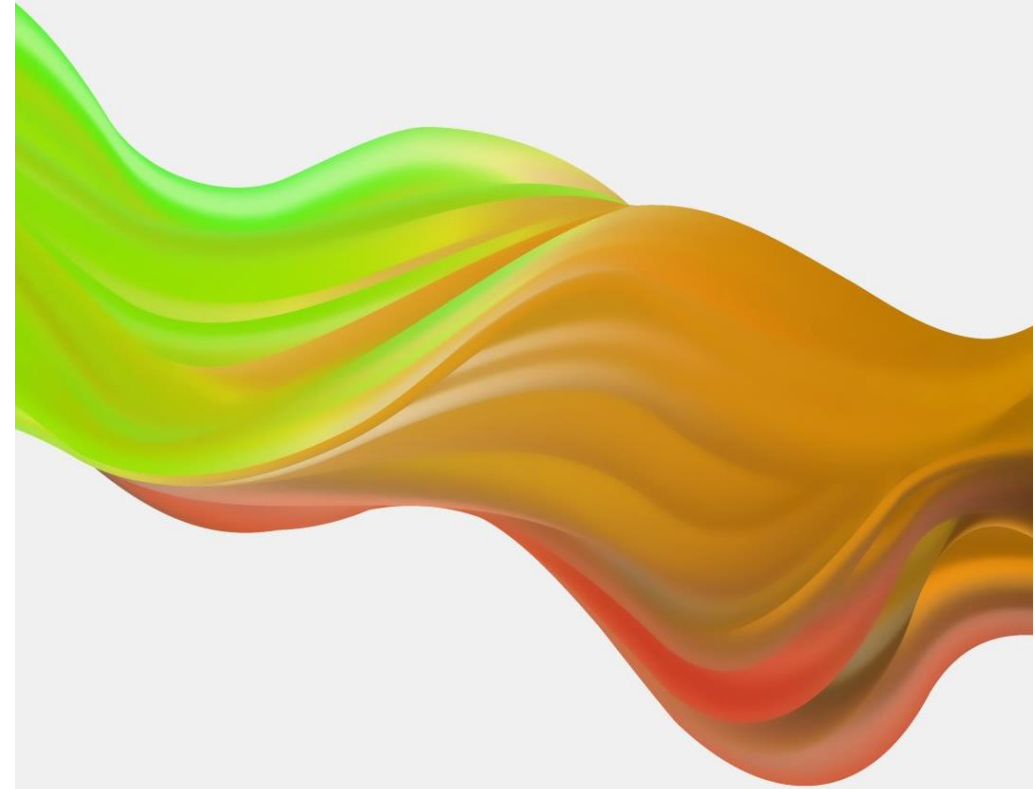


Computer Programming

MENG2020

Copyright © Dr. Maryam Davoudpour

Copyright © Dr. Greg Reese, Miami University. Dr. Amos Gilat,
John Wiley & Sons, Inc.



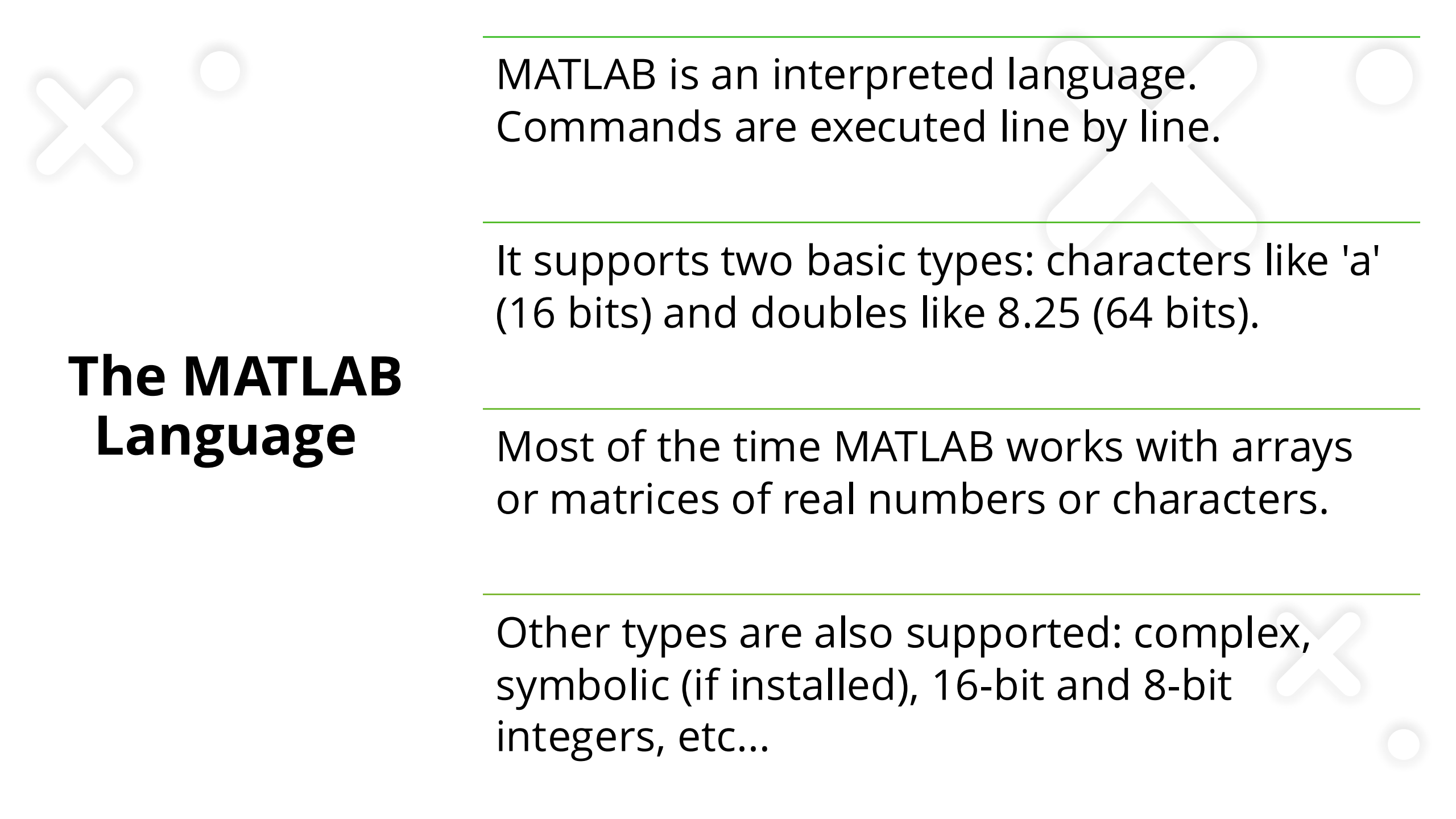


What Is MATLAB?

MATLAB has two different methods for executing commands: interactive mode and batch mode.

In interactive mode, commands are typed (or cut and pasted) into the *command window*.

In batch mode, a series of commands is saved in a text file with a **.m** extension. The batch commands in a file are then executed by typing the name of the file at the MATLAB command prompt.



The MATLAB Language

MATLAB is an interpreted language.
Commands are executed line by line.

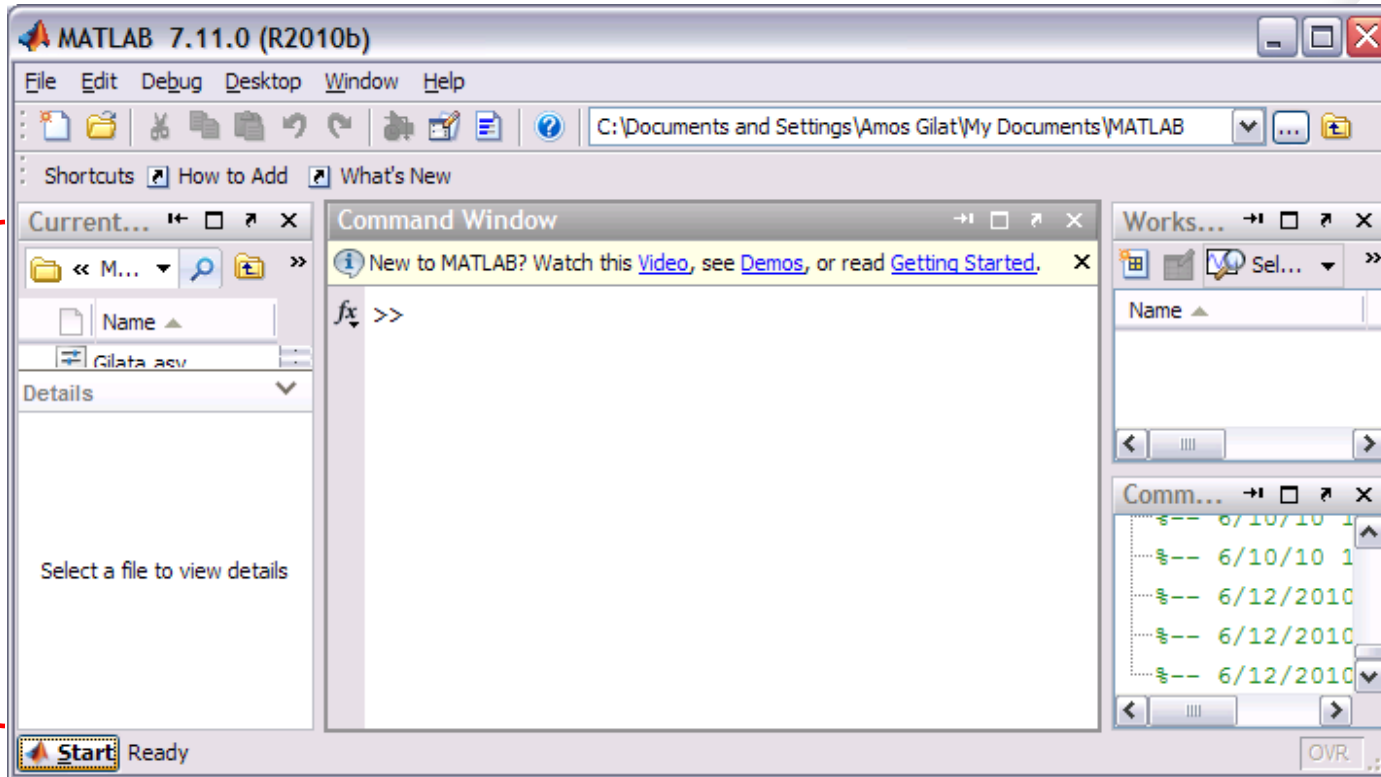
It supports two basic types: characters like 'a' (16 bits) and doubles like 8.25 (64 bits).

Most of the time MATLAB works with arrays or matrices of real numbers or characters.

Other types are also supported: complex, symbolic (if installed), 16-bit and 8-bit integers, etc...

The MATLAB Desktop

Current Folder Window



Workspace Window

Command History Window

Command Window

The Figure Window

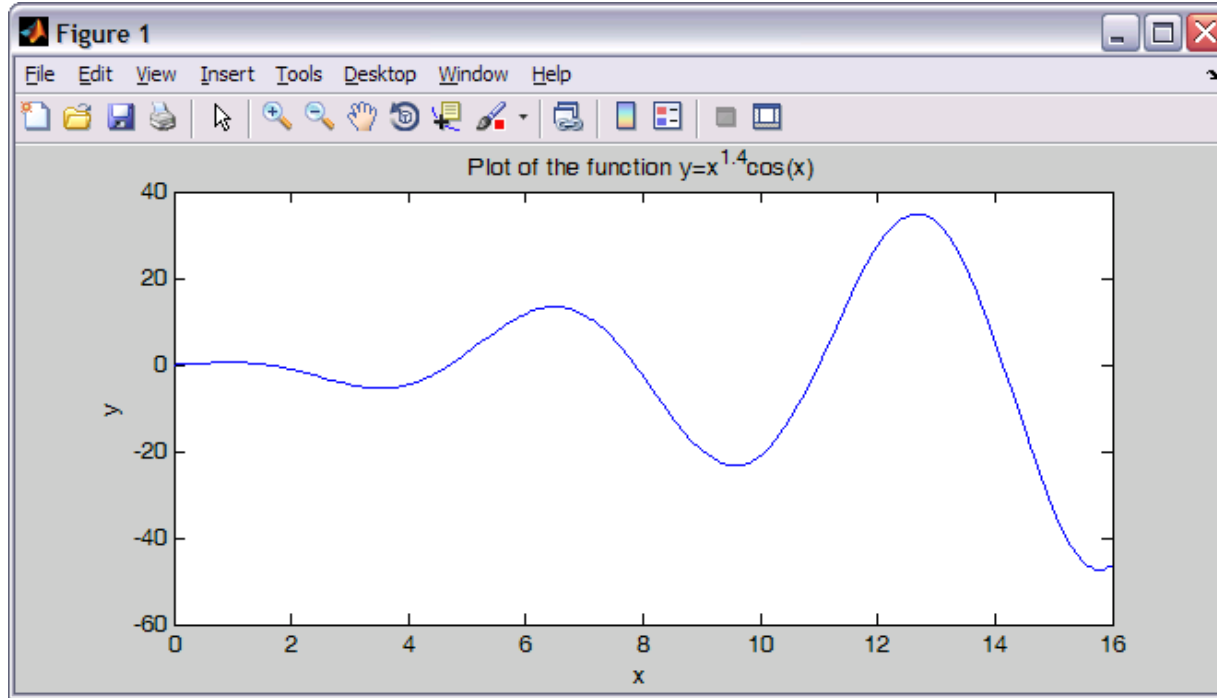
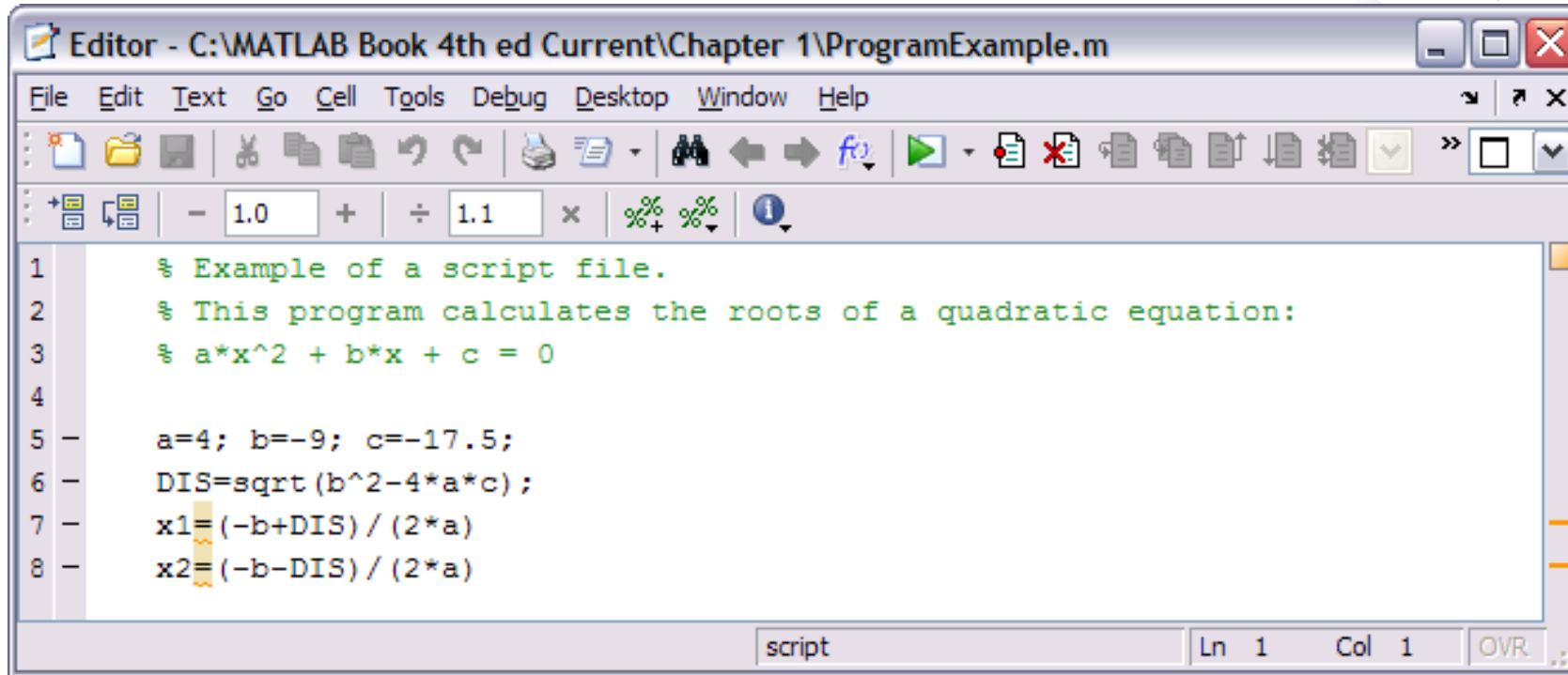


Figure Window

The Figure Window opens automatically after any command that draws a graph.

The Editor Window



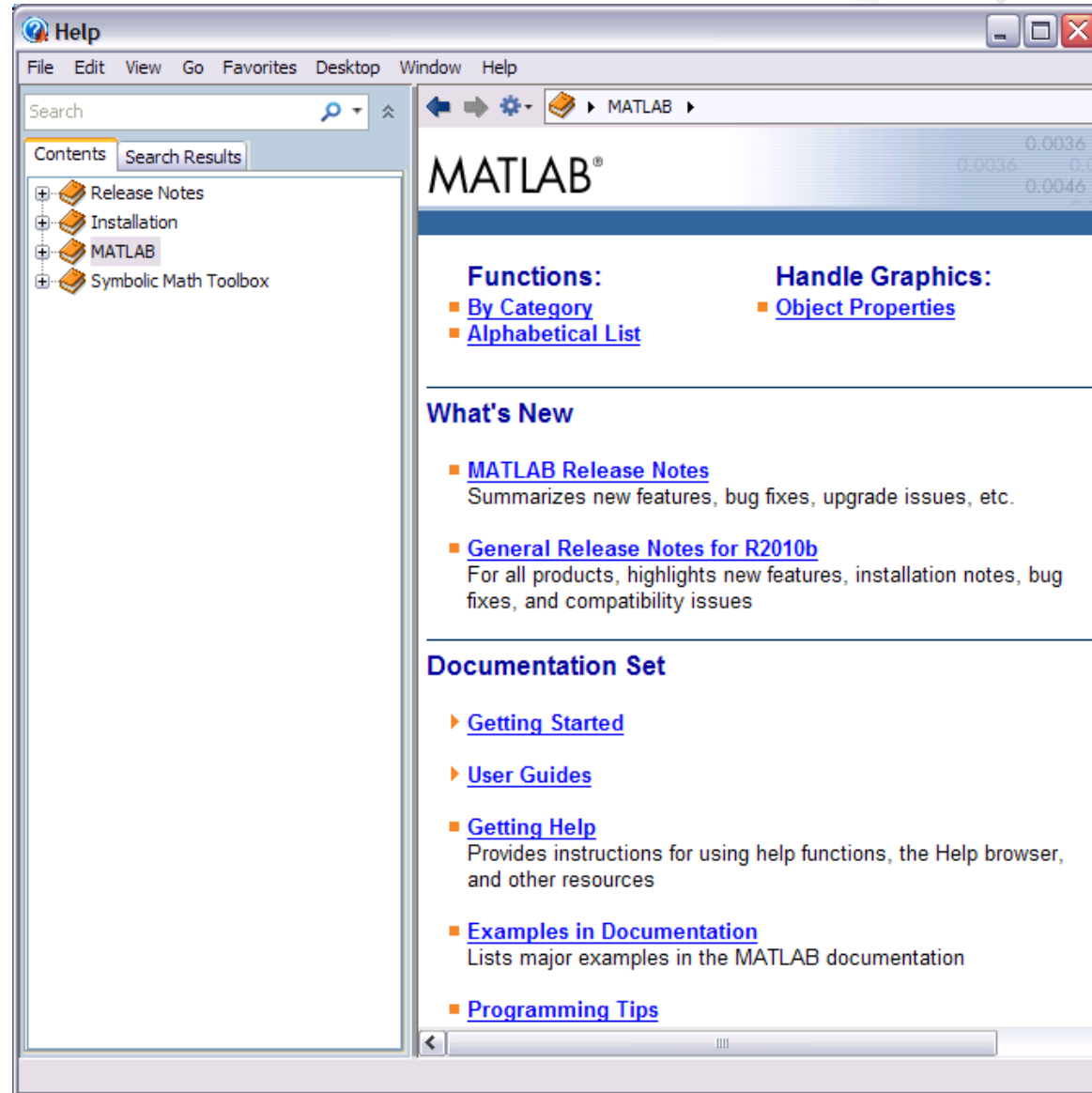
Editor/Debugger Window

Use the editor/debugger window to write and debug MATLAB scripts. Open with the Edit command.

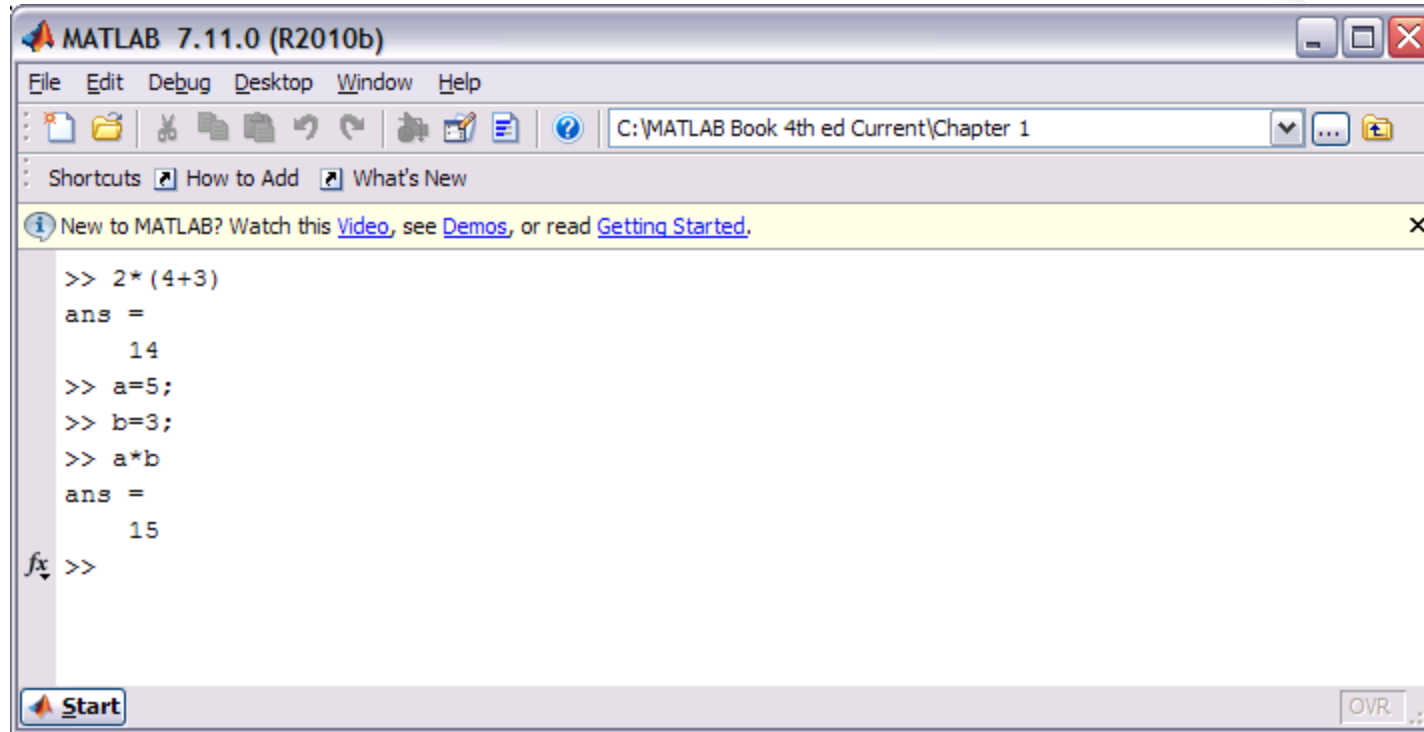
The Help Window

Get the help window from the Help menu in any MATLAB window.

(may vary according to version)



The Command Window



At the prompt (>>), type in a MATLAB command. Press the ENTER key.

MATLAB displays the result in the command window, followed by a prompt.

Repeat the process.

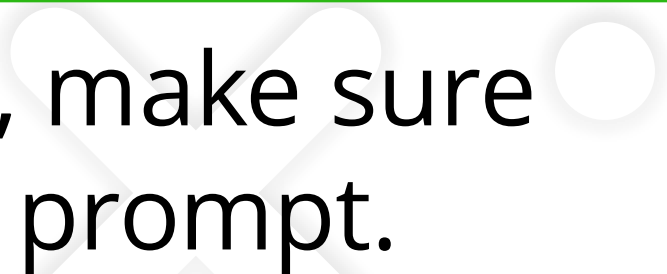


Notes on the Command Window

To start a command, make sure cursor is next to the prompt.

MATLAB won't respond until you press ENTER. It then executes only the last command.

Commands before the last one may still be visible, but MATLAB doesn't execute them.





Notes on the Command Window



You can type several commands in the same line by putting a comma between commands. (makes the program harder to read though, so it is not recommended).

If a command is too long to fit on one line, you can continue to the next line by typing ellipsis (3 periods, i.e., ...) and then pressing ENTER.



Navigating the Command Window

When cursor is in the bottom (current) command line:

← key moves the cursor one character to the left.

→ key moves the cursor one character to the right.

↑ key recalls the preceding command

↓ key recalls a previous command if it follows.



Navigating the Command Window

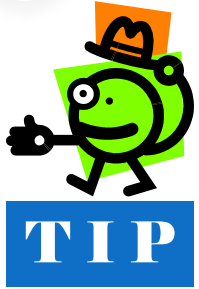
The PAGE-UP key moves up to previous commands in a window-size at a time.

The PAGE-DOWN key moves down to previous commands in a window-size at a time.

The BACKSPACE (Delete on Mac) key deletes one character to the left of the cursor.

The DELETE (fn-Delete on Mac) key deletes one character to the right of the cursor.





To quickly execute a previous command but with small changes:

Recall the command with the up and down arrow keys.

Use the left and right arrow keys to move to characters to be altered.

Use BACKSPACE (Mac: Delete) or DELETE (Mac: fn-Delete) to remove old characters, then type new characters.

Press ENTER to execute the modified command.

Command Syntax

Semicolon (;)

- When typed at end of a command, it suppresses the output. (only the prompt is displayed at the next line)

Useful for preventing display of large outputs

Used much more in scripts (see Section 1.8 of text book)

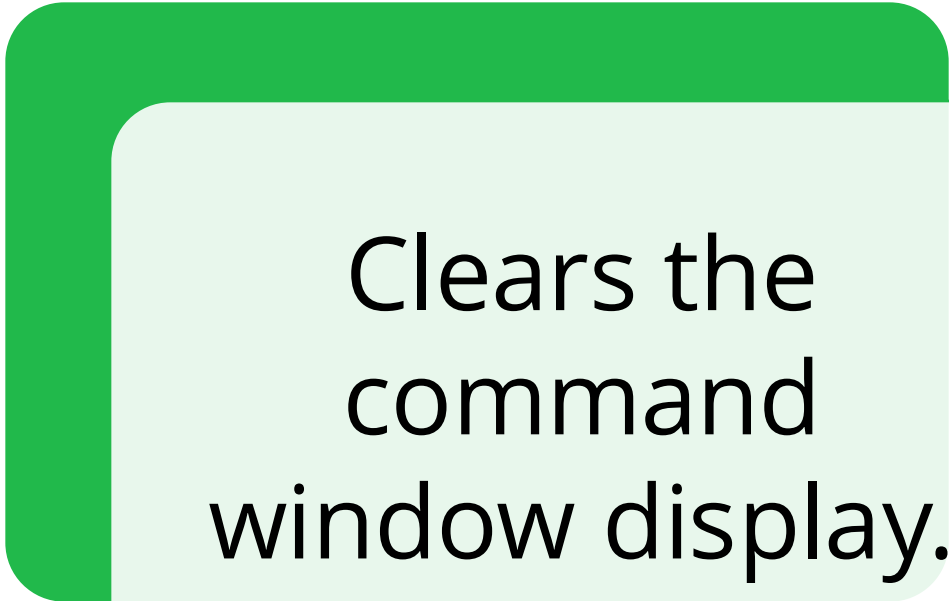
Command Syntax

Percent sign (%)

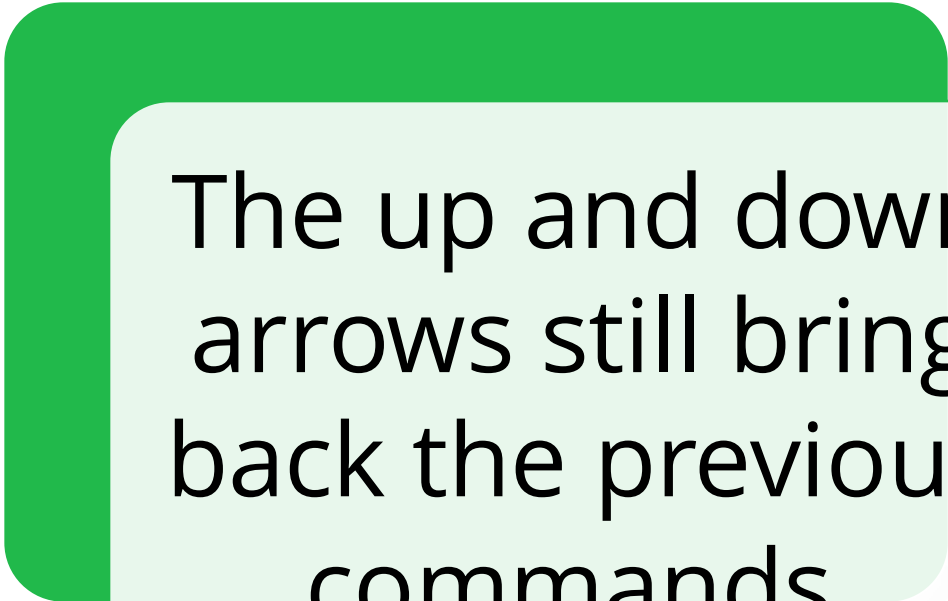
- When typed at beginning of a line, MATLAB treats the line as a *comment* and doesn't execute the line.

Also used much more in scripts.

The clc Command



Clears the
command
window display.



The up and down
arrows still bring
back the previous
commands.



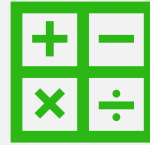


The Command History Window

Shows the previous commands, including the ones from previous MATLAB sessions.

Double-clicking on a command puts it in the command window and executes it.

You can drag a command to the command window, make changes in it, then execute it.



In this lesson we will only discuss arithmetic with *scalars* (single numbers).

Arithmetic Operations



We can do arithmetic directly on numbers (like a calculator).



Or, we can store numbers in variables (containers).

Arithmetic Operators

Symbols for arithmetic operations are:

Operation	Symbol	Example
Addition	+	$5 + 3$
Subtraction	—	$5 - 3$
Multiplication	*	$5 * 3$
Right division	/	$5 / 3$
Left division	\	$5 \setminus 3 = 3 / 5$
Exponentiation	^	$5 ^ 3$ (means 5 to the power of 3 = 125)

Note: Left division is rarely used with scalars.

Arithmetic Operations Precedence Order

Order in which MATLAB does arithmetic:

Precedence	Mathematical Operation
First	Parentheses. For nested parentheses, the innermost are executed first
Second	Exponentiation
Third	Multiplication, division (equal precedence)
Fourth	Addition and subtraction



Arithmetic Operations Precedence Rules



The MATLAB precedence order is:



The same as most calculators.



The same as doing arithmetic by hand.



For multiple operations of same precedence, MATLAB goes left to right.



You can change the order by using parentheses.



CLASS EXAMPLES WERE SOLVED



× Parentheses



- + Use only ()
- + { } and [] mean something different
- + MATLAB does not assume operators

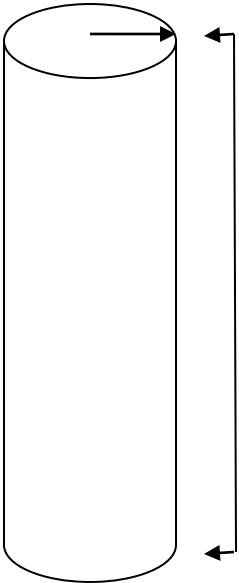
$5 * (3+4)$ not $5(3+4)$



EXAMPLE

Find the surface area of a cylinder

$r = \text{radius}$ $h = \text{height}$
 $r = 5$ $h = 10$



$=$ πr^2 $+$ $2\pi r * h$

πr^2

$$SA = 2\pi r^2 + 2\pi rh = 2\pi r(r + h)$$

Using MATLAB as a Calculator

Type in a mathematical expression.

Press the **Enter** key.

MATLAB displays the answer in the command window as
ans = followed by the result.

Example:

```
>> 2 * 3  
      ans = 6
```

Your display may appear on more than one line and have blank lines between the text.



Formatting the Display

You can control the display of numbers with the **format** command.

Once a command is entered, the format stays the same until another **format** command is issued.

The default format is fixed point with four digits to the right of the decimal point. *fixed-point* means decimal point always between one's-digit and one-tenth's digit.

The **format** command only affects the display of numbers. MATLAB always computes and saves numbers in full precision.

MATLAB Display Formats

Matlab command	Display	Comments
Format short	3.1416	default
Format long	3.14159265358979	16 digit
Format short e	3.1416e+000	5 digit plus exponent
Format long e	3.14159265358979e+000	16 digit plus exponent
Format bank	3.14	2 decimal places
Format +	+	positive
Format rat	355/113	ratio
Format hex	400921fb54442d18	hexadecimal

MATLAB Built-In Functions

MATLAB expressions can include functions. You can think of a *function* as a black box that, in general, takes inputs, does some computations with them, and produces outputs.



MATLAB Built-In Functions

A function has a name.

It can have zero or more *arguments* (inputs).

It can produce zero or more outputs.

This is a typical use of such functions:

output name argument

↓ ↓ ↓

`y = sqrt(x)`

MATLAB Built-In Functions

A function's arguments can be numbers, variables (containers or receptacles), or expressions involving numbers, variables, or other functions.

Examples:

`sqrt(64)` argument is the number 64

`sqrt(a)` argument is the variable named a

`atan(y/sqrt(3^2+y^2))`

argument is an expression that contains numbers (2 and 3), a variable (y), and a function call (sqrt)

Elementary Math Functions

$\text{sqrt}(x)$ – square root

$\text{nthroot}(x,n)$ – n th real root

$\text{exp}(x)$ – e to the power of x

$\text{abs}(x)$ – absolute value

$\log(x)$ – natural log (base e)

$\log_{10}(x)$ – log base 10

$\text{factorial}(x)$ – $x!$

See text book for more details.

Elementary Trig Functions

$\sin(x)$ – sine (x in radians)

$\text{sin d}(x)$ – sine (x in degrees)

$\cos(x)$ – cosine (x in radians)

$\text{cos d}(x)$ – cosine (x in degrees)

$\tan(x)$ – tangent (x in radians)

$\text{tan d}(x)$ – tangent (x in degrees)

$\cot(x)$ – cotangent (x in radians)

$\text{cot d}(x)$ – cotangent (x in degrees)

See text book for more details.

$$\text{angle in degrees} = \text{angle in radians} \cdot \frac{180^\circ}{\pi}$$

$$1 \text{ rad} = 1 \cdot \frac{180^\circ}{\pi} \approx 57.2958^\circ$$

Rounding Functions

`round(x)` – round to nearest integer

`fix(x)` – round toward zero (transforms a double into an integer)

`ceil(x)` – round toward infinity

`floor(x)` – round toward minus infinity

`rem(x, y)` – remainder after x is divided by y (also called modulus)

`sign(x)` – returns 1 if x is positive, -1 if x is negative, zero if x is zero

See text book for more details.

What Is a Variable?

A *variable* is a name that is assigned a value. Think of it as a container or receptacle that has a name and contains some value.

Once assigned with a meaningful value, you can use variables in expressions, functions, and MATLAB statements and commands.

Assigning Values to Variables



= (the equal sign) is MATLAB's *assignment operator*.



It evaluates the expression on its right side and stores the resulting value in the variable on its left side

Assigning Values to Variables

= (the equal sign) is MATLAB's *assignment operator*. It evaluates the expression on its right side and stores the resulting value in the variable on its left side

Q?

Create the variable called `a` and store the value 3 in it

Assigning Values to Variables

= (the equal sign) is MATLAB's *assignment operator*. It evaluates the expression on its right side and stores the resulting value in the variable on its left side

Create the variable called a and store the value 3 in it

```
>> a = 3
```

MATLAB acknowledges that it has created a and set it to 3

```
a =
```

```
3
```

Assignment Operator

Example:

```
>> a = 3
```

← Create a variable and store a number in it

```
a =
```

```
3
```

```
>> b = 10 * a + 5
```

← Create a variable and store the value of an expression made up of a variable, numbers, and addition and multiplication

```
b =
```

```
35
```

How Assignment Works



Think of `=` as meaning “assign to” or “store in” but it is not meaning “equals”!

Why? A left arrow would make more sense but that symbol does not exist on keyboards! So we use `=` instead.

$x = x + 6$ has no meaning in math because it implies that $0 = 6$. But `x = x + 6` is perfectly fine in MATLAB because it means “take whatever is in `x` (the `x` on the right side), add 6 to that and store the result back into `x` (the `x` on the left side which really is the same container (variable) named `x`)”.

How Assignment Works



$x = x + 6$ has no meaning in math because it implies that $0 = 6$.

But $x = x + 6$ is perfectly fine in MATLAB because it means “take whatever is in x (the x on the right side), add 6 to that and store the result back into x (the x on the left side which really is the same container (variable) named x)”.

Assignment Operator Examples

Example:

```
>> x = 3;
```

← ; at end prevents MATLAB from displaying value of x.

```
>> x = x + 6
```

← takes what's in x (3), adds 6 to it to get 9, then stores 9 back into x.

```
x =
```

```
9
```

← now the value of x is 9

```
>> x = 2 * x
```

← takes what's in x (9), multiplies it by 2 to get 18, then stores 18 back into x

```
x =
```

```
18
```

← now the value of x is 18

Assignment Operator Warning



A variable must have a value before you use it in an expression.

```
>> x = 3;
```

```
>> x + 2
```

```
ans = ↙
```

```
5
```

Q?

>> x + y

Assignment Operator Warning



```
>> x + y % assume y is undefined
```

```
??? Undefined function or  
variable 'y'
```

This is a comment (follows %, ignored by MATLAB)

The Value of a Variable

```
>> x = 3;
```

```
>> y = 10 * x;
```

```
>> z = y ^ 2;
```

```
>> y
```

```
y =
```

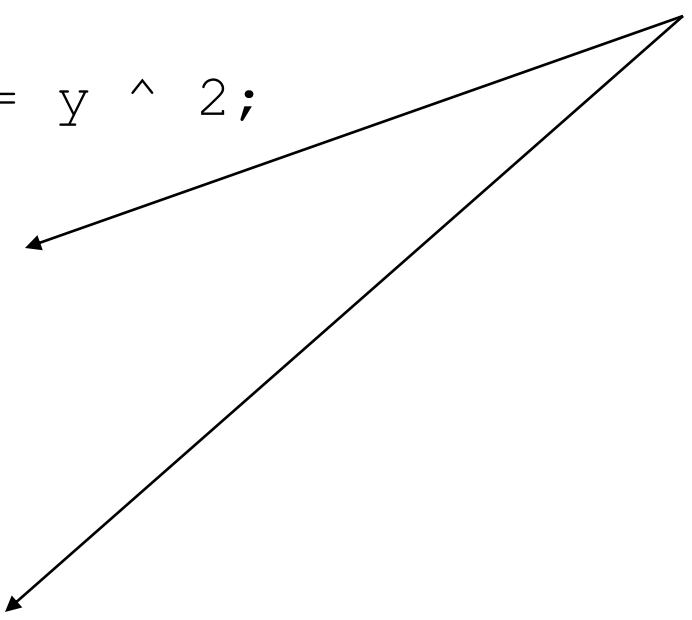
```
30
```

```
>> z
```

```
z =
```

```
900
```

To find out the value of a variable,
just type it and press ENTER.



Multiple Assignments

You can do multiple assignments on one line by separating them with a comma or semicolon.

If you use a semicolon, the resulting value is not displayed but the assignment happens nonetheless.

```
>> a=12, B=4; C=(a-B)+40-a/B*10
```

```
a =
```

```
12
```

```
C =
```

```
18
```

Changing the Value of a Variable

To change the value of a variable, just assign it the new value.

```
>> ABB=72;
```

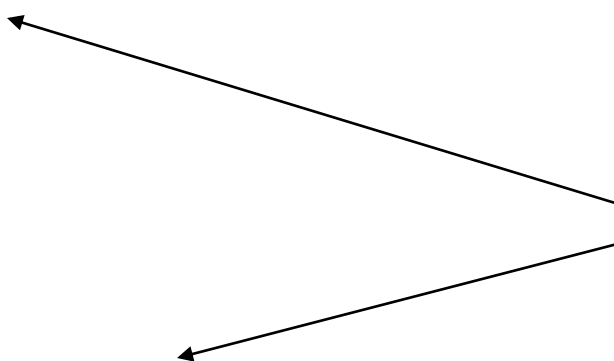
```
>> ABB=9;
```

```
>> ABB
```

```
ABB =
```

```
9
```

Only the last value is kept!
One value at a time in a scalar
variable. The original value of 72 is
gone!



$$r=5$$

$$\text{area}=\pi*r^2$$

$$r=10$$

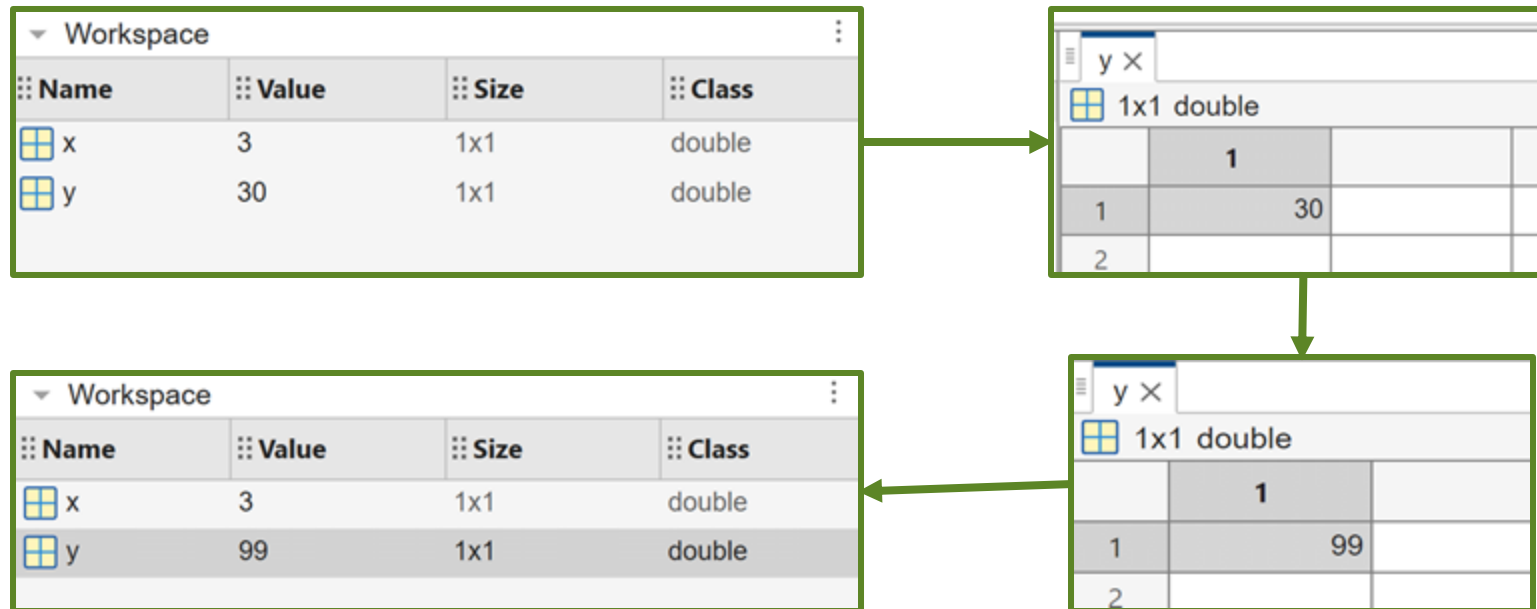
$$\text{surface_area}=4*\pi*r^2$$

$$r=2$$

$$\text{volume}=\frac{4}{3}*\pi*r^3$$

Workspace Area & Variable Editor

- The **workspace** area shows all the variables and their content.
- We can modify the variables on the fly using the variable editor (double-click on a variable to access).



Workspace Area

The **workspace** area shows all the variables and their content.

We can modify the variables on the fly using the variable editor (double-click on a variable to access).

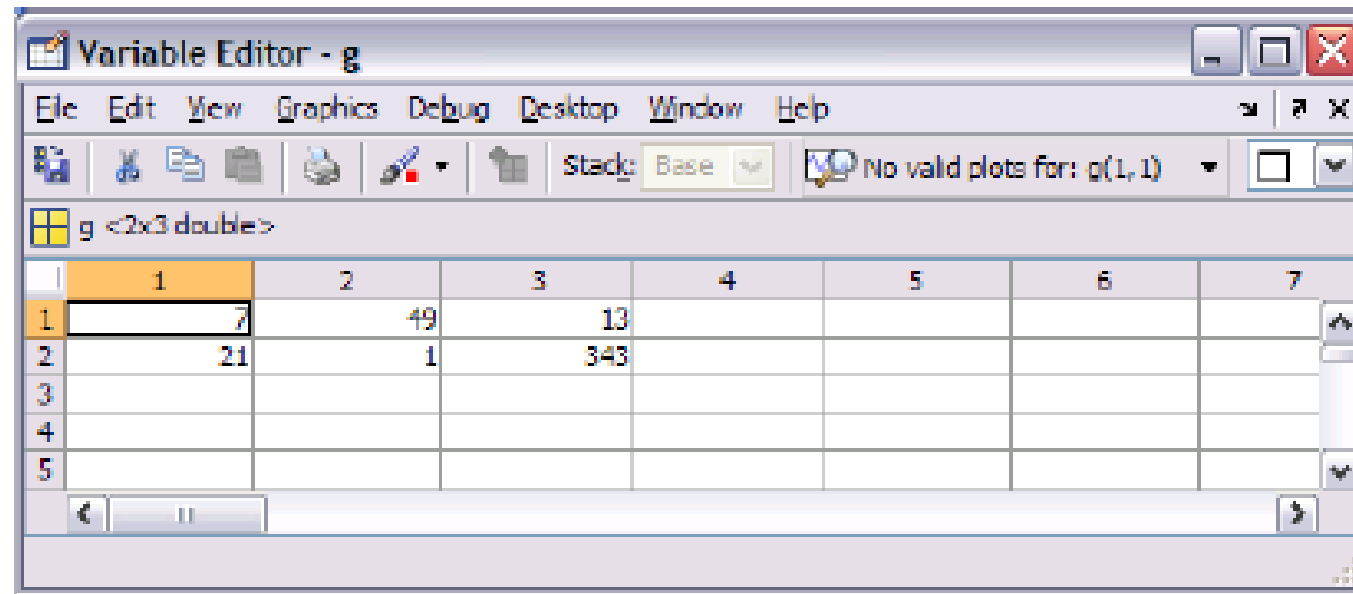


Figure 4-2: The Variable Editor Window.

Variables and Functions

You must define a variable (give it a value) before you can use it in an argument of a function.

An undefined variable is like an empty container. It is useless.

Q?

>> sqrt(x)

Q?

```
>> sqrt( x ) % assume x undefined
```

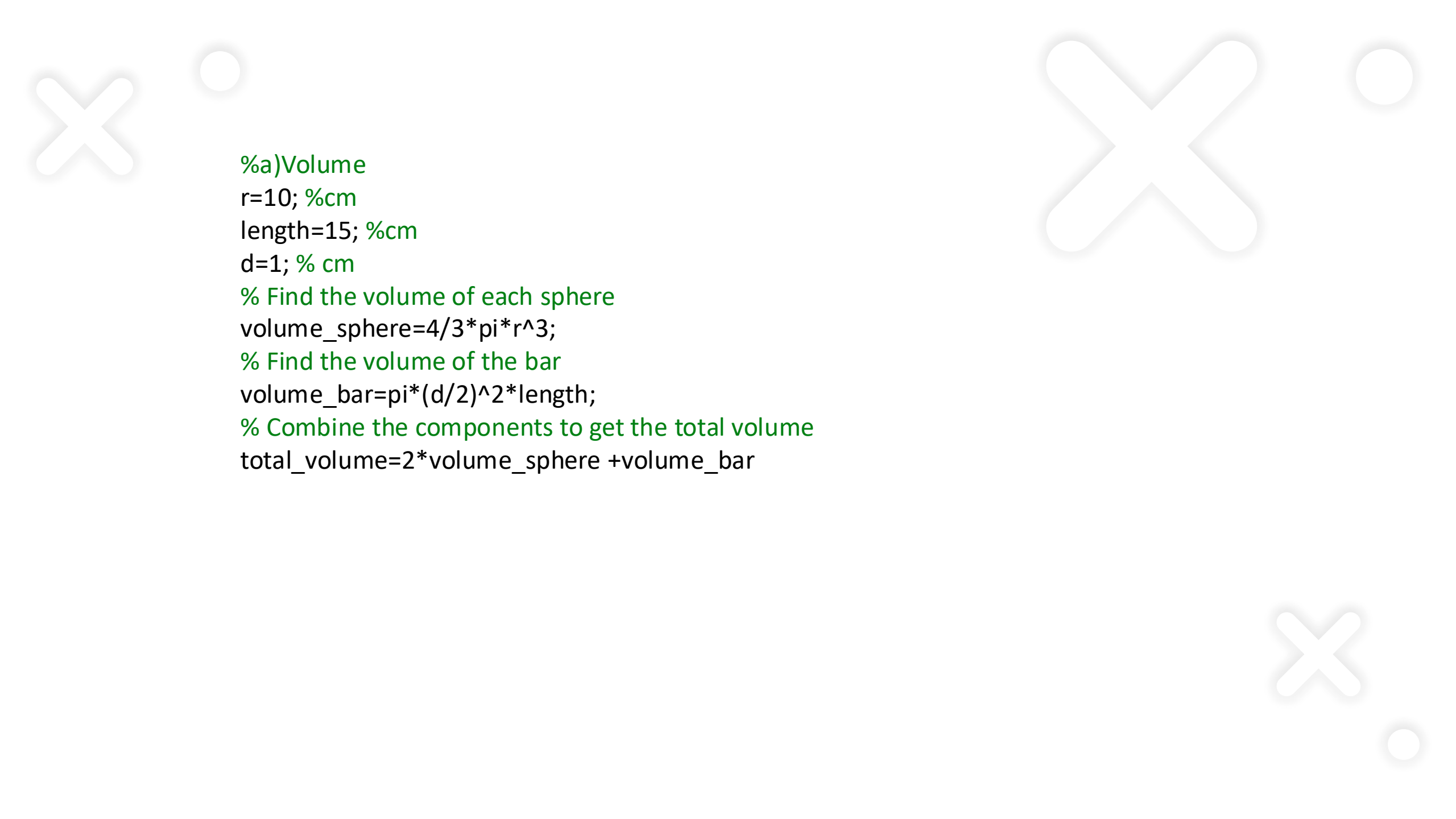
```
??? Undefined function or variable 'x'
```

```
>> x = 144;
```

```
>> sqrt( x )
```

```
x =
```

```
12
```



```
%a)Volume
r=10; %cm
length=15; %cm
d=1; % cm
% Find the volume of each sphere
volume_sphere=4/3*pi*r^3;
% Find the volume of the bar
volume_bar=pi*(d/2)^2*length;
% Combine the components to get the total volume
total_volume=2*volume_sphere +volume_bar
```

Name ▲	Value
 area	78.5398
 d	1
 f_c	1
 F_c	1.0343
 fs	10
 length	15
 n	6
 r	10
 surface_area	1.2566e+03
 T	0.1000
 total_volume	8.3894e+03
 volume	33.5103
 volume_bar	11.7810
 volume_sphere	4.1888e+03

Variable Name Rules

Must begin with a letter.

Can be up to 63 characters long.

Can contain letters, digits, and underscores (_).

Can't contain punctuation, e.g., period, comma, semicolon.

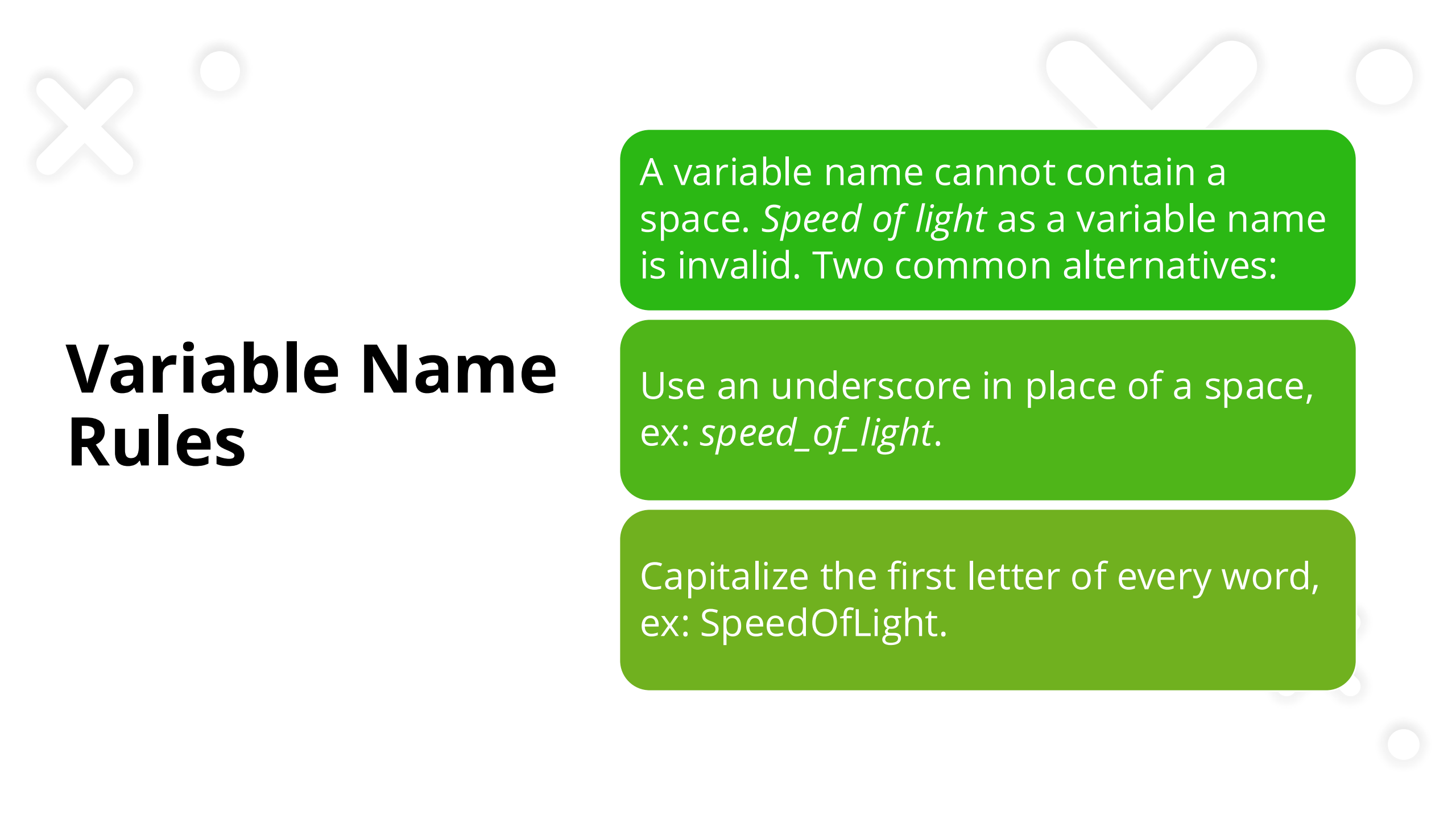
Avoid using the name of a built-in function as the name of a variable, e.g., don't call a variable exp or sqrt.



Variable Name Rules

MATLAB is *case-sensitive*, and does not consider an upper-case letter in a variable name to be the same as its lower-case counterpart,

SPEED, Speed, speed, and SpEEd are four different variable names!



Variable Name Rules

A variable name cannot contain a space. *Speed of light* as a variable name is invalid. Two common alternatives:

Use an underscore in place of a space, ex: *speed_of_light*.

Capitalize the first letter of every word, ex: *SpeedOfLight*.

Variable Name Rules

MATLAB Keywords

A *keyword* or *reserved word* is a word that has special meaning to MATLAB. There are 20 keywords.

`break, case, catch, classdef, continue, else, elseif, end, for, function, global, if, otherwise, parfor, persistent, return, spmd, switch, try, while`

They appear in blue when typed in the editor/debugger window (type *iskeyword* to see the list).

They can't be used as variable names!

Predefined Variables

There are a few predefined variables in MATLAB that you will use often.

Both **i** and **j** are defined to be the square root of -1

pi is defined as the usual 3.1416.

inf is used for infinity

eps is used for the smallest difference between two numbers or 2^{-52} (epsilon).

*These variables **can be** assigned other values but it is not recommended.*

ans: the value of the last expression that was not assigned to a variable.

NaN or **nan**: not-a-number. Used to express mathematically undefined values, such as 0/0.

○ The clear

- + You can remove a variable from MATLAB's memory by clearing it (`clear x`).
- + Clearing a predefined variable will restore its default value (`clear pi`).
- + `clear` by itself removes all variables.